

Relevant Source Code Documentation

Attendance & Face Recognition System — *detect.py*

This document covers the core functions directly used by or supporting `refresh_with_detections()`, the main per-frame detection and tracking pipeline.

1. Main Detection Pipeline

`refresh_with_detections()`

Source: `detect.py` : line 3239

The central per-frame function that orchestrates the entire recognition and tracking workflow.

```
def refresh_with_detections(frame, rgb, frame_idx):
    """
    IMPROVED FOLLOWING - green boxes follow people even when far away
    RELAXED             - less strict protection zones for better tracking
    ENHANCED            - adaptive thresholds for far-away tracking
    FIXED               - tracking through obstacles / partial occlusion
    RETAINED            - prevents identity swapping with more flexibility
    """
    global tracks, locked_tracks, pending_confirmations, KNOWN_FACE_ENCODINGS_ARRAY
    global detectionStopped, current_fps, skip_frame_counter, student_presence_tracker

    SKIP_FACTOR = 2    # process every 2nd frame
    if frame_idx % SKIP_FACTOR != 0:
        if tracks:
            for tr in tracks:
                tr["last_seen"] = frame_idx
            return

    # Steps executed each processed frame:
    # 1. Emergency face-encoding reload if DB is empty
    # 2. Face detection (InsightFace + threshold fallback for side faces)
    # 3. Body detection (detect_bodies)
    # 4. ByteTrack body tracking + ReID feature extraction
    # 5. Movement prediction & protection-zone management for locked tracks
    # 6. ReID-based locked-track update
    # 7. Face recognition (cosine similarity, section validation, anti-swap)
    # 8. Pending-confirmation → lock promotion + attendance marking
    # 9. Track list cleanup & confidence decay
```

How it works

- Skips odd frames (`SKIP_FACTOR=2`) to reduce CPU load; still updates `last_seen` on skipped frames.
- On the first frame or when the face-encoding array is empty, triggers an emergency reload from the database.
- Runs InsightFace detection; if no face is found, retries with a lower detection threshold (0.2) for side/profile faces.
- Passes body detections to ByteTrack for stable cross-frame IDs, then extracts per-body ReID feature vectors.

- For each locked identity, updates a movement-history ring-buffer (last 20 positions), predicts the next bounding-box location, and builds adaptive protection zones (inner / outer / prediction) whose margins scale with pixel distance from frame centre.
- Updates locked track positions via ReID distance matching with up to 4 validation stages (spatial, body-signature, temporal consistency, overlap).
- Runs cosine-similarity face recognition against the known-face array. Applies distance-adaptive thresholds (0.57–0.63 frontal, slightly lower for profile), section-membership validation, ambiguity gap check (≥ 0.12 required), and facial-landmark sanity check.
- Accumulates per-identity confirmation frames; promotes to a locked track once CONFIRMATION_FRAMES_REQUIRED frames with sufficient average similarity are collected, then calls `mark_attendance()`.
- Applies confidence decay ($\times 0.85/\text{frame}$) to unlocked tracks and removes stale Unknown tracks after 30 frames.

2. Face Detection & Recognition Helpers

`iou()`

Source: `detect.py` : line 1016

```
def iou(box1, box2):
    """Intersection-over-Union for two (x1,y1,x2,y2) boxes."""
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])
    inter = max(0, x2-x1) * max(0, y2-y1)
    area1 = (box1[2]-box1[0]) * (box1[3]-box1[1])
    area2 = (box2[2]-box2[0]) * (box2[3]-box2[1])
    union = area1 + area2 - inter
    return inter / union if union > 0 else 0.0
```

Used extensively in `refresh_with_detections()` for protection-zone checks, face-to-body matching, and ReID overlap validation.

`load_known_faces_from_db()`

Source: `detect.py` : line 1475

```
def load_known_faces_from_db():
    """
    Loads all known face encodings from the database into global lists.
    Calls rebuild_known_faces_array() to build the numpy matrix.
    Returns True on success.
    """
    global known_face_encodings, known_face_names, known_face_ids
    global known_face_types, KNOWN_FACE_ENCODINGS_ARRAY
    # ... queries DB, parses 512-d embeddings, populates globals ...
```

- Called on frame 1, or any time `KNOWN_FACE_ENCODINGS_ARRAY` is `None`/empty.
- Also called in aggressive-reload mode (every 50 frames) when the array remains empty after the first load attempt.

rebuild_known_faces_array()

Source: `detect.py` : line 1396

```
def rebuild_known_faces_array():
    """
    Converts known_face_encodings list → normalised numpy array
    (KNOWN_FACE_ENCODINGS_ARRAY) for fast cosine-similarity dot-product.
    Sets KNOWN_FACE_ENCODINGS_NORMALIZED = True.
    """
```

Pre-normalising the array means recognition reduces to a single `np.dot()` call per frame.

get_current_section_student_ids()

Source: `detect.py` : line 1190

```
def get_current_section_student_ids():
    """
    Returns a set of student IDs enrolled in the currently active section.
    Used in refresh_with_detections() to reject recognised faces that
    belong to a different section ("Unknown - Wrong Section").
    """
```

Guards against cross-section false positives. Called once per accepted recognition match (medium-confidence branch skips it to save a DB round-trip).

load_multi_angle_encodings()

Source: `detect.py` : line 4806

```
def load_multi_angle_encodings():
    """
    Returns a dict { student_id: [list_of_512d_encodings] }
    covering frontal + profile angles for each enrolled student.
    Used when face_orientation != "front" to extend the candidate pool.
    """
```

- Invoked inside `refresh_with_detections()` when a side/profile face is detected.
- Combines primary frontal encodings with stored multi-angle encodings and runs the same dot-product similarity search across the larger pool.

3. Body Detection & Tracking

detect_bodies()

Source: `detect.py` : line 6102

```
def detect_bodies(frame, frame_idx):
    """
```

```
Runs YOLO (or equivalent) person detection on `frame`.
Returns list of { "box": (x1,y1,x2,y2), "confidence": float }.
Filters out detections smaller than min-size thresholds.
"""
```

- Called in Step 2 of `refresh_with_detections()` to get raw body bounding boxes every processed frame.
- Results are fed directly into ByteTrack via `sv.Detections`.

extract_reid_features()

Source: `detect.py` : line 6989

```
def extract_reid_features(frame, body_box):
    """
    Crops the body region from `frame` and extracts a colour-histogram
    or deep-feature Re-Identification vector.
    Returns a 1-D numpy array, or None on failure.
    """
```

Called for every ByteTrack detection. The resulting vector is stored in the track dict and blended into the locked-track ReID model with exponential moving average ($\alpha=0.90-0.95$ depending on distance).

calculate_reid_distance()

Source: `detect.py` : line 7035

```
def calculate_reid_distance(feet1, feat2):
    """
    Returns a scalar distance between two ReID feature vectors.
    Lower = more similar. Threshold used in refresh: 0.25.
    """
```

calculate_box_distance()

Source: `detect.py` : line 7022

```
def calculate_box_distance(box1, box2):
    """
    Euclidean distance between the centres of two bounding boxes.
    Used in the spatial-constraint stage of ReID matching.
    """
```

4. Attendance & Track Management

mark_attendance()

Source: `detect.py` : line 2454

```
def mark_attendance(name, id, type, session_id=None):
```

```
"""
Records an attendance event for a student or faculty member.
Called exactly once per identity when a pending confirmation is
promoted to a locked track.
"""
```

- Writes to the DB with a late-status calculation (compare attendance timestamp vs session start time).
- Idempotent within a session — duplicate calls for the same id are ignored.

cleanup_pending_confirmations()

Source: `detect.py` : line 6970

```
def cleanup_pending_confirmations(current_frame_idx, timeout_frames=10):
    """
    Removes entries from pending_confirmations whose last_seen frame
    is older than timeout_frames. Called at the top of every
    processed frame to prevent stale confirmation state.
    """
```

cleanup_locked_tracks()

Source: `detect.py` : line 2965

```
def cleanup_locked_tracks(current_frame, lock_timeout_frames):
    """
    Removes locked tracks whose last_seen is older than
    lock_timeout_frames. Frees associated history, prediction,
    and signature dicts.
    """
```

5. Utility & Supporting Functions

get_current_session_id()

Source: `detect.py` : line 6916

```
def get_current_session_id():
    """Returns the active session ID, or None if no session is running."""
```

add_unknown_face()

Source: `detect.py` : line 20637

```
def add_unknown_face(face_crop, face_encoding, track_id=None):
    """
    Saves an unrecognised face crop + embedding to the unknown-faces
    store for later manual enrolment by admin.
    Deduplicates by embedding similarity before storing.
    """
```

expand_box()

Source: detect.py : line 1009

```
def expand_box(x1, y1, x2, y2, w, h, scale=EXPAND_BOX_RATIO):  
    """Enlarges a bounding box by `scale`, clamped to frame dimensions."""
```

get_current_class()

Source: detect.py : line 15113

```
def get_current_class():  
    """  
    Returns metadata for the currently active class/section,  
    used to scope student lookups in get_current_section_student_ids().  
    """
```

detect_face_orientation()

Source: detect.py : line 4847

```
def detect_face_orientation(face_kps, face_width):  
    """  
    Determines face orientation (front / left / right profile)  
    from InsightFace keypoints.  
    Returns a string: "front", "left", or "right".  
    """  
    left_eye, right_eye, nose = face_kps[0], face_kps[1], face_kps[2]  
    eye_distance = abs(right_eye[0] - left_eye[0])  
    if face_width > 0:  
        eye_ratio = eye_distance / face_width  
        if eye_ratio < 0.4: # profile face  
            eye_midpoint = (left_eye[0] + right_eye[0]) / 2  
            if nose[0] < eye_midpoint - (face_width * 0.1):  
                return "right"  
            elif nose[0] > eye_midpoint + (face_width * 0.1):  
                return "left"  
    return "front"
```

- Called inside refresh_with_detections() to decide whether to use standard frontal recognition or the multi-angle encoding pool.
- Profile faces use a slightly lower similarity threshold (-0.01) to compensate for the reduced embedding overlap between frontal-trained models and side views.

normalize_embedding()

Source: detect.py : line 4872

```
def normalize_embedding(embedding):  
    """  
    L2-normalises a raw 512-d InsightFace embedding so that  
    cosine similarity reduces to a simple dot product.  
    Returns the normalised array, or None if norm == 0.  
    """  
    norm = np.linalg.norm(embedding)  
    if norm > 0:  
        return embedding / norm
```

```
return None
```

- Applied to every live face embedding before the `np.dot()` similarity search.
- Also applied to `KNOWN_FACE_ENCODINGS_ARRAY` on-the-fly if `KNOWN_FACE_ENCODINGS_NORMALIZED` is `False` at recognition time.

calculate_late_status()

Source: `detect.py` : line 3203

```
def calculate_late_status(attendance_time, session_start, threshold_minutes):  
    """  
    Compares attendance_time against session_start + threshold_minutes.  
    Returns "on_time", "late", or "absent" string used when  
    writing the attendance record inside mark_attendance().  
    """
```

- Called indirectly via `mark_attendance()` when a pending confirmation is locked.
- `threshold_minutes` is read from `get_session_threshold()` which pulls the value set by the faculty for the active session.

is_face_scan_active()

Source: `detect.py` : line 6927

```
def is_face_scan_active():  
    """  
    Returns True if the face-scan window is currently open  
    (i.e. the faculty has started a scan session).  
    refresh_with_detections() checks this as a top-level gate –  
    recognition and locking are suppressed when scan is inactive.  
    """
```

- Acts as the master on/off switch for the recognition pipeline within `refresh_with_detections()`.
- Paired with `detectionStopped` global flag; both must allow processing for attendance to be recorded.

6. Call-Graph Summary

The following lists every function called directly inside `refresh_with_detections()`, grouped by responsibility:

Face DB

- `load_known_faces_from_db()` — emergency/startup reload
- `rebuild_known_faces_array()` — rebuild normalised numpy matrix

Detection

- `face_analysis.get(rgb)` — InsightFace face detection

- `detect_bodies(frame, ...)` — YOLO body detection

Tracking

- `byte_tracker.update_with_detections()` — ByteTrack body IDs
- `extract_reid_features()` — per-body ReID vector
- `calculate_reid_distance()` — ReID similarity score
- `calculate_box_distance()` — spatial movement constraint
- `iou()` — bounding-box overlap

Recognition

- `load_multi_angle_encodings()` — profile-face encodings
- `get_current_section_student_ids()` — section membership filter

Attendance & State

- `mark_attendance()` — record attendance event
- `cleanup_pending_confirmations()` — expire stale candidates
- `get_current_session_id()` — active session guard
- `add_unknown_face()` — enrol unrecognised face